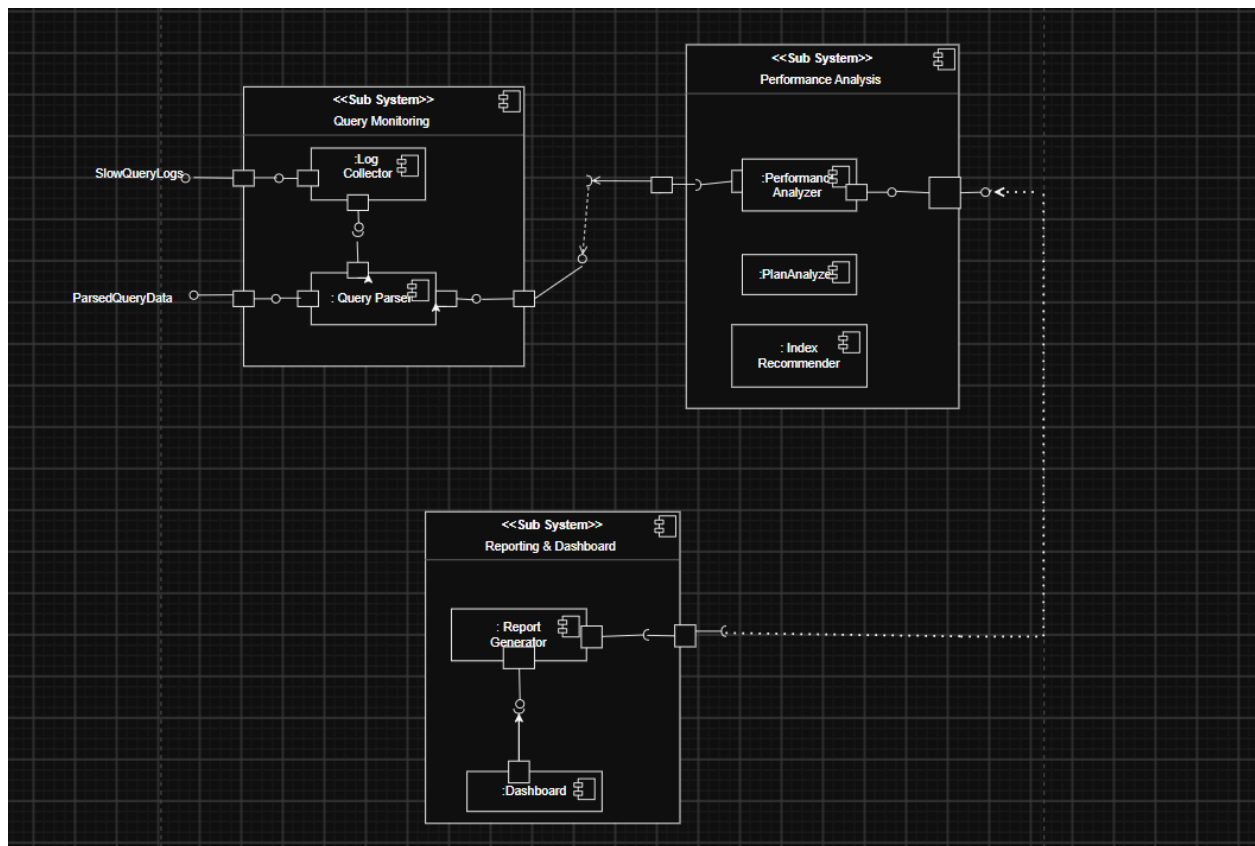


Lab 3: Component Modelling & Architectural Pattern Selection

Shreya Narayana Bhat
PES2UG24CS483

Database Query Performance Profiler

Component Diagram



Database Query Performance Profiler

Component Diagram – Architecture Justification

1. Components

Query Monitoring

- **Log Collector** – collects slow query logs.
- **Query Parser** – reads and extracts useful information from the logs.

Performance Analysis

- **Performance Analyzer** – checks which queries are slow or expensive.
- **Plan Analyzer** – checks the SQL execution plan.
- **Index Recommender** – suggests missing or useful indexes.

Reporting & Dashboard

- **Report Generator** – creates performance reports and weekly summaries.
- **Dashboard** – displays the results to the users.

2. Main Connections

Log Collector → Query Parser

Query Parser → Performance Analyzer

Performance Analyzer → Index Recommender

Report Generator → Dashboard

3. Interfaces

- **Provided interface (○)**: shows what a component provides.
- **Required interface ()**: shows what a component needs.

- **Assembly connector:** connects the required interface to the provided interface.
- **Dependency (<<use>>):** shows that one component uses another component.

For example:

Query Parser provides Parsed Query Data, which is required by **Performance Analyzer** and **Plan Analyzer**.

Index Recommender provides Optimization Results, which is required by **Report Generator**.

4. Architecture Selection

Architectural Choice

We chose Pipe-and-Filter Architecture for the Database Query Performance Profiler.

The data passes through different stages, such as collecting, parsing, analyzing, recommending indexes, and generating reports.

Reason 1

Each component has a **separate job**, so the system is easier to understand and maintain.

Reason 2

The architecture is suitable for processing **many slow-query logs step by step**.

Security Advantage

The database logs are processed through separate components, so the Dashboard does not need direct access to the raw database logs.

Performance Benefit

Different stages can process the data separately, which helps the system handle a **large number of query logs efficiently**.